Changelog

Changelog

All notable changes to this project are documented here. Format follows Keep a Changelog and Semantic Versioning. Dates are ISO 8601.

[1.4.0] - 2026-07-23

Infrastructure and shipping-readiness release — no change to the model or the analysis since 1.3.0.

Added

- `.github/workflows/ci.yml` — CI that runs the Python test suite (on Python 3.10/3.11/3.12) and the JS/Python parity harness on every push and pull request. Closes the standing "a standard that is not checked is only a preference" gap for this project (it previously had tests but no CI).
- `tests/js_parity.js` — the JS/Python parity check is now a committed, runnable script (previously an ad-hoc extraction), so CI can run it and it can't silently rot. It checks the exact Tier 1/Tier 2 values, the inverse-normal and Bonferroni-band reference values, and the paired-SE <= independent-SE invariant. Partially mitigates risk #8.
- `docs/README.md` — a Diátaxis documentation index and entry point.

Changed

- Registered this project in the cross-project audit script (`../portfolio/build/check_projects.py`) and made that script's `ROOT` portable (derived from its own location, with a `PROJECTS_ROOT` env override) instead of a hardcoded absolute sandbox path. The audit now reports this project passing all 12 required-artifact checks and version consistency (changelog 1.4.0 <-> white paper Version 1.4).

Notes

- Still local-only until the git lock is cleared and the repo is pushed: the CI workflow is written but will not *run* until the project is on GitHub, and the app is not yet deployed to a live URL. "Three places agree" is not yet satisfied — see the handoff steps in the session.

[1.3.0] - 2026-07-23

Fixed

- `engine/dd_wager.py` — **the Daily Double wager-selection statistics were wrong beneath the earlier fix.** A second, referee-style review (prompted by treating the project as a thesis submission) found that the v1 fix (version 1.1.0) computed the wager-comparison standard error with the *independent* two-sample formula `sqrt(se_w^2 + se_best^2)`, while the simulator uses **common random numbers** (CRN), under which candidate wagers are strongly positively correlated. The independent formula discards exactly the variance reduction CRN provides, over-widens the "plateau," and makes the engine systematically **under-bet** — for the originally reported scenario `[4200,6000,3400]`@0.42 at 3000 trials it recommended $100 where the CRN-correct analysis recommends $4,200.

  - *The fix (two coupled changes):* (1) the comparison standard error is now the **paired** per-trial difference between a candidate wager and the empirical best wager, which respects CRN; (2) because ~k wagers are each compared against the single empirical best (a multiple-comparisons-with-the-best problem, Hsu 1996), the band is a multiplicity-adjusted Bonferroni value `z* = Phi^{-1}(1 - alpha/k)` (default `alpha = 0.05`) rather than a per-comparison `z`.
  - *Verified on both ends:* a flat scenario recommends $0 stably across seeds and neighbouring confidences (right call — the surface is truly flat), and a clear-signal (guaranteed-correct) scenario recommends a positive $1,200 stably across seeds (it does **not** collapse to $0).
  - Ported to `app/index.html` (`ddAnalyze`, plus `pairedStandardError`, `inverseNormalCdf`, `bonferroniZ`) and re-verified under Node.
  - Full method and evidence: `docs/reviews/statistical-review-2026-07-23-v2.md`. Risk register: risk #9 annotated as superseded, new risk #10 added and resolved.

Changed

- `engine/dd_wager.py` `DDRecommendation` gained `comparison_standard_errors` and `z_score_used`; `analyze()`'s `z_score` now defaults to `None` (compute the adaptive Bonferroni band) with a new `alpha` parameter, and `z_score` can still be passed explicitly as an override.
- Corrected two overstatements found in the same review, without deleting the prior text: the engine no longer claims Monte Carlo "monotonicity guarantees" (win probability is not monotone in the wager, because opponents' modeled FJ wagers depend on the player's score — risk, wording fixed), and the white paper now says the Tier 2 enumeration is "exact enumeration in double precision (no sampling error)" rather than implying exact rational arithmetic (risk #13).

Added

- New documented limitations from the v2 review, each with a risk-register row: reported Daily Double win probability is selection-biased upward (winner's curse, risk #11); Tier 2 assumes opponent correctness is independent across players though it is really positively correlated, so "everyone busts" upset probabilities are conservative floors (risk #12); Python/JS PRNG divergence means the app's Monte Carlo numbers are only distributionally comparable to the reference engine (extends risk #8).
- 3 new engine tests (33 total, up from 30): `test_paired_comparison_se_is_never_larger_than_independent_formula` (the invariant that proves the CRN pairing is actually used), `test_bonferroni_band_widens_with_more_candidate_wag` (hand-checked against standard-normal quantiles), and `test_recommends_a_positive_wager_when_the_signal_is_cl`

[1.2.0] - 2026-07-23

Added

- Diátaxis technical documentation, written in ASD-STE100 (Simplified Technical English — short sentences, active voice, one meaning per word), completing a required standing artifact:
  - `docs/tutorial/getting-started.md` — a full walkthrough with known, reproducible expected numbers for one Final Jeopardy and one Daily Double scenario, checkable against this page.
  - `docs/how-to/get-a-wager-recommendation.md`, `docs/how-to/practice-with-quiz-mode.md`, `docs/how-to/maintain-the-project.md` — task-oriented guides for using the app and for maintaining the project (tests, JS/Python parity check, PDF regeneration, changelog/risk-register discipline).
  - `docs/reference/engine-api.md` — every public function and dataclass in `engine/`, with parameter and return-value tables, checked field by field against the actual source (not written from memory).
  - `docs/explanation/tiers-explained.md` — a plain-language companion to the white paper's Tier 1/Tier 2 discussion, for a reader who wants the idea before the math.
- PDF companions for all six new docs, plus five previously un-PDF'd scaffold files from the 0.1.0–1.1.0 releases (`README.md`, `SECURITY.md`, `CONTRIBUTING.md`, `CLAUDE.md`) — closing the gap noted in this

changelog's own standing PDF-companion requirement. `LICENSE` and `.gitignore` are intentionally not PDF'd (boilerplate/config, not prose documentation).
- `docs/deck.pptx`: a 12-slide plain-English deck (title, the two moments that carry stakes, Safe Bet vs. Smart Bet with worked examples, the Daily Double simulation and its plateau fix told as a before/after story, the 3-way "both leaders bust" scenario, an app tour, a proven-vs-placeholder honesty slide, and a closing slide linking `white-paper.md`), completing the last required standing artifact for this project. Built with `pptxgenjs`, validated with the pptx skill's schema/relationship checker (all checks passed), and visually reviewed slide by slide (no overflow, no overlaps).

Notes

- `docs/how-to/maintain-the-project.md` references the cross-project artifact-audit script at `../portfolio/build/check_projects.py`. That script's hardcoded project list does not yet include this project — flagged there rather than silently assumed to work.
- Test suite unchanged this release (still 30/30 passing); this release is documentation-only.
- Four scratch image files used for the deck's visual QA (`docs/deck-preview-tmp.jpg`, `docs/qa-tmp-1.jpg`, `docs/qa-tmp-2.jpg`, `docs/qa-tmp-3.jpg`) could not be removed from this folder due to a sandbox limitation (the same one noted in the 0.2.0 release, affecting the local git lock file) — my tools cannot delete files inside the synced Projects folder. They are QA scratch, not project artifacts; safe to delete by hand.

[1.1.0] - 2026-07-23

Fixed

- `engine/dd_wager.py` **(Daily Double engine)** — the wager selection rule compared Monte Carlo win-probability estimates using a fixed `1e-9` tolerance, treating noisy estimates as exact. Reported by Will: a 2-point change in stated confidence (42% → 44%) swung the recommended wager from $350 to $4,200 for identical scores. Fixed by tracking each candidate's standard error (`sqrt(p(1-p)/trials)`) and recommending the smallest wager that is statistically indistinguishable from the best estimate (within a configurable number of combined standard errors), reporting the full set of such wagers as a "plateau." `trials_per_wager` default raised 800 → 3,000. Full writeup: `docs/reviews/statistical-review-2026-07-23.md`. Risk register: new risk #9 (resolved).
- `engine/fj_bestresponse.py` **(Final Jeopardy Tier 2)** — the wager search was indifferent between a wager that only ties an opponent's worst case and one that beats it outright, whenever both hit the same tie-inclusive win probability. Reported by Will: "the smart bet results in a tie if both are correct that's dumb," seen at 100% confidence where the strict win costs nothing extra. Fixed by adding strict-win probability (no tie) as a secondary tiebreaker: among wagers tied on tie-inclusive win probability, prefer whichever wins outright more often, then the smallest wager. This **reverses a documented conclusion** from `docs/white-paper.md` section 5.4 and the CHANGELOG entry below dated 2026-07-23 (version 0.2.0) — see the correction note added directly in white paper section 5.4 for what changed and why; the prior conclusion is not deleted, only superseded.
- Both fixes ported from the Python engine into the JavaScript port, `app/index.html` (`outcomeProbabilities`/`fjTier2`, `ddAnalyze`). Verified by extracting the `<script>` block and running it under Node against 8 hand-derived/regression values from the Python test suite — all matched.

Changed

- `app/index.html`: replaced the dense per-wager bar-chart visualization (Will: "i hate the bar graph") with a single SVG curve of win probability against wager size — one gold dot marking the recommendation, a shaded band showing the Daily Double engine's statistical plateau where one applies, and a one-line plain-language caption explaining what the shape means. Used for the Daily Double, Final Jeopardy, and Quiz tabs.
- `app/index.html` footer now links to Will's portfolio (https://willhoop.github.io/) instead of printing a raw local file path.

- Confirmed (not a bug) via a new hand-derived test, `test_third_place_can_win_when_both_leaders_bust`, that the existing exact-expectation math in `fj_bestresponse.py` already correctly models the 3-way case Will asked about: a trailing player winning because both leading opponents bet big and miss. Matches the ~8.3% (1/12) the app displayed for his exact reported scores (`[4000, 9000, 10000]`).

Notes

- Document/project versioning is unified starting this release: the version stamped on `docs/white-paper.md` (the project's primary artifact) and the top `CHANGELOG.md` entry now use the same number (1.1.0 / "Version 1.1"). Earlier releases used two separate, inconsistent numbering tracks (project semver 0.1.0–0.4.1 vs. document "Version 1.0") — not corrected retroactively, just aligned going forward.
- Test suite: 30/30 passing, up from 26. 4 new tests this release: `test_standard_error_matches_binomial_formula`, `test_plateau_contains_recommended_wager_and_is_sorted`, and `test_small_confidence_change_no_longer_ca` (all in `tests/test_dd_wager.py`), plus `test_third_place_can_win_when_both_leaders_bust` (in `tests/test_fj_bestresponse.py`). `test_analyze_recommends_the_smaller_tie_safe_wager` was rewritten in place as `test_analyze_prefers_strict_win_over_gratuitous_tie` to match the corrected conclusion (see Fixed, above) rather than deleted.
- `RISK_REGISTER.md`, `docs/white-paper.md`, and this changelog were all updated in this same pass, per the project's living-documentation standard. Affected PDFs in `build/` regenerated in this pass too (see below).

[0.4.1] - 2026-07-23

Added

- PDF companions for every Markdown doc in the project, generated with `pandoc` (pdflatex engine) into `build/`: `white-paper.pdf`, `wagering-theory-research.pdf`, `adr-0001-data-source-strategy.pdf`, `adr-0002-opponent-model-assumptions.pdf`, `risk-register.pdf`, `changelog.pdf`. Requested by Will as a standing preference: every doc gets a PDF version going forward.

Notes

- The Markdown files remain the source of truth; the PDFs in `build/` are generated copies. If a `.md` file changes, its PDF companion needs to be regenerated in the same pass, the same way a version bump accompanies a code change – otherwise the PDF silently goes stale. Nothing automates this yet (no CI, per earlier notes), so it's a manual step to remember.
- Two Unicode minus signs (U+2212) inside code blocks broke the first PDF build attempt (pdflatex has no glyph for it outside math mode) – fixed by replacing them with plain hyphens in the source Markdown.

[0.4.0] - 2026-07-23

Added

- `docs/white-paper.md`: the finalized, citable white paper, consolidating the research memo, both ADRs, and the engine math into one document with worked examples and full references – one of the project's standard required artifacts. Requested by Will to study the theory in one place.
- White paper section 4.4 formalizes an insight from Will's question about "irrelevant rivals": a rival whose worst-case ceiling never exceeds the binding opponent's ceiling has no effect on the Tier 1 wager, since the math only ever looks at the single largest threat. This was already true of the implementation; it just wasn't written down anywhere before. Logged as a future UI enhancement (not built) in the research memo: surfacing this explicitly in the app (e.g., "Rival 2 doesn't affect this wager").

Notes

- `docs/explanation/wagering-theory-research.md` remains the running research log; the white paper is the polished, "send this to someone" version and should be read first.

[0.3.1] - 2026-07-23

Fixed

- `app/index.html`: the "Daily Doubles still hidden after this one" dropdown showed both "None" and "One more out there" even when Single Jeopardy (which has exactly one Daily Double total) was selected by default on page load. The round-based restriction only ran on a button *click*, so the default selection never triggered it. Fixed by running the same update function once on load, and by matching the static markup to the default round. Caught by Will from a screenshot of the dropdown open while Single Jeopardy was selected.

[0.3.0] - 2026-07-23

Added

- `app/index.html`: interactive, single-file browser trainer. Jeopardy-themed UI with four tabs:
  - **Daily Double** – enter scores, round, money left on the board, whether another Daily Double is still hidden, and a confidence slider ("how do you feel about this category?"); get a recommended wager and a bar chart of win probability across candidate wagers.
  - **Final Jeopardy** – same idea, side-by-side Safe Bet (Tier 1) vs. Smart Bet (Tier 2) comparison.
  - **Quiz Me** – randomly generated DD or FJ scenarios, styled as clue cards; type in a wager, get graded (A+ to D) against the optimal play, with a running score/streak persisted in the browser (localStorage) and a confetti burst at a 3-streak.
  - **How This Works** – plain-language explanation of Safe Bet vs. Smart Bet, with pointers back to the ADRs and Python engine.
- The app is a hand-written JavaScript port of `engine/fj_wager.py`, `engine/fj_bestresponse.py`, `engine/simulate.py`, and `engine/dd_wager.py`. Verified for parity against the Python test suite's hand-derived values (all of `tests/test_fj_wager.py`'s cases plus three `fj_bestresponse.py` cases plus the DD wager-bound cases reproduced exactly in Node before shipping) – see the verification transcript in this session; not wired into an automated CI parity check yet.

Notes

- The JS port is **not** part of the automated pytest suite – it was spot-checked once against known values, not continuously tested. If the Python engine changes, the JS port must be updated by hand and re-checked; nothing currently catches drift between them automatically. Tracked as a new risk below.
- Still no README, LICENSE, CI, or working git in this folder (see 0.2.0 notes – unchanged this release).

[0.2.0] - 2026-07-23

Added

- `engine/fj_bestresponse.py`: Tier 2 Final Jeopardy engine. Given a player's own confidence and an opponent wagering model, computes the exact (not simulated) expected win probability for each candidate wager and recommends the smallest wager achieving the best one. Ties for first count as wins, matching the actual show rule – this is why it can recommend a smaller wager than Tier 1 in some cases (see `test_analyze_recommends_the_smaller_tie_safe_wager`).
- `engine/simulate.py`: shared Monte Carlo game-continuation model (regular clues, optional future Daily Doubles, Final Jeopardy via Tier 1 logic). All simplifications are named in the module docstring.
- `engine/dd_wager.py`: Daily Double wagering engine. Searches candidate wagers using Monte Carlo rollouts of the rest of the game, with common random numbers across candidates within each trial to keep the comparison fair and reduce sampling noise.

- `docs/adr/0002-opponent-model-assumptions.md`: documents that the opponent-behavior constants used by Tier 2 and the DD engine are illustrative placeholders, not fitted values, and what would need to happen to replace them.
- 17 new tests across `tests/test_fj_bestresponse.py`, `tests/test_simulate.py`, and `tests/test_dd_wager.py` (26 total in the project now). Where exact hand derivation wasn't possible (Monte Carlo output), tests check invariants instead: legal wager bounds, behavior at the extremes of confidence, determinism given a fixed seed, and probabilities staying in [0, 1].
- Risk register: resolved risk #4 (Tier 1 turned out not to need pairwise decomposition), added risks #5-7 covering the new placeholder constants, Monte Carlo sampling noise, and simulator simplifications.

Notes

- Local git repository hit a sandbox limitation: my tools can't delete files inside the synced Projects folder, and a git lock file got stuck after the first commit, which now blocks every further git command in this folder from my side. The one commit that succeeded is intact (`5c83b5b`). To keep using git here, delete the `.git` folder inside `jeopardy-wagering` yourself and run `git init` again – this works fine on your actual machine, the restriction is specific to my sandbox.
- GitHub push was never possible in this session regardless – the GitHub connector isn't authorized. Connect it via `claude mcp` or `/mcp` (or your claude.ai connector settings) if you want me to push in a future session.
- Given the above, "three places agree" doesn't apply yet: everything here exists locally only.
- Tier 2 and the DD engine are directionally correct but use uncalibrated placeholder opponent-behavior numbers (ADR-0002) – don't quote their win-probability outputs as validated statistics yet.

[0.1.0] - 2026-07-23

Added

- Research memo on Final Jeopardy and Daily Double wagering theory (`docs/explanation/wagering-theory-research.`
covering two-player and three-player heuristics, the Watson/Tesauro et al. game-theoretic approach, and a draft expected-value model for Daily Double wagers.
- ADR-0001: data source strategy — use published Tesauro et al. statistics for shipped artifacts, restrict the jwolle1 historical dataset to internal validation only, due to its reuse restrictions. Accepted 2026-07-23.
- Initial risk register with four open/tracked risks.
- Project folder scaffold (`docs/`, `engine/`, `build/`, `data/`, `tests/`, `assets/`).
- `engine/fj_wager.py`: Tier 1 (guarantee/worst-case) Final Jeopardy wagering engine. Computes lockout detection, minimum cover wager, and lock-tie wagers for 1, 2, or 3 players. Does not yet model opponent confidence or win probability – see the research memo, section 3.3, for the planned Tier 2 layer.
- `tests/test_fj_wager.py`: 9 tests, expected wagers derived by hand from the classic cover-bet formula, not copied from the module's output. All passing locally.
- `.gitignore` for Python build artifacts.

Notes

- FJ was built before Daily Double because its guarantee-based math is fully determined (no confidence/probability inputs needed), per Will's call to do the "more fixed" piece first.
- Written locally only. No git repository or GitHub remote is set up yet, so nothing has been pushed and no CI workflow can run. README, LICENSE, SECURITY.md, and CONTRIBUTING.md are still pending.
- Tier 2 (probabilistic best-response engine, Daily Double engine) not started.